



首页

AI Coding

沸点

课程

直播

活动

AI刷题

APP

插件

探索稀土掘金



创作者中心



会员

Swift底层原理学习笔记

逍遥归来 2025-06-30 21 阅读9分钟

Trac IDE 已支持 Doubao-Seed-1.6 模型，推理更快、准确度更高、稳定性更强

笔记主要记录Swift和OC底层原理差异的地方，OC的底层原理之前的笔记有详细记录。
课程是逻辑教育的，视频基本只看了总结部分，然后结合网上已有笔记进行的重点梳理。

Swift 进阶一：类，对象，属性

类、对象

- Swift对象的内存结构 **HeapObject**，有两个属性：一个是 **Metadata**，一个是 **RefCount**，默认占用 **16** 字节大小，就是对象中没有任何东西也是 **16** 字节。
- OC中实例对象的本质是结构体，是以objc_object为模板继承的，其中有一个isa指针，占 **8** 字节。
- Swift比OC中多了一个 **refCounted** 引用计数大小，也就是多了8字节。

- getClassObject** 函数是根据 **kind** 获取 **object** 类型
- 如果 **kind**（理解为isa指针）是 **Class** 类型，则将当前的 **Metadata** 强转成 **ClassMetadata**，而 **ClassMetadata** 是 **TargetClassMetadata** 根据类型的别名，其中 **TargetClassMetadata** 结构：

```
struct TargetClassMetadata : public TargetAnyClassMetadata<Runtime> {  
    using StoredPointer = typename Runtime::StoredPointer;  
    using StoredSize = typename Runtime::StoredSize;  
  
    TargetClassMetadata() = default;  
    constexpr TargetClassMetadata(const TargetAnyClassMetadata<Runtime> &base,  
        ClassFlags flags,  
        ClassIvarDestroyer &ivarDestroyer,  
        StoredPointer size, StoredPointer addressPoint,  
        StoredPointer alignMask,  
        StoredPointer classSize, StoredPointer classAddressPoint)  
        : TargetAnyClassMetadata<Runtime>(base),  
        Flags(flags),  
        InstanceAddressPoint(addressPoint),  
        InstanceSize(size), InstanceAlignMask(alignMask),  
        Reserved(0), ClassSize(classSize), ClassAddressPoint(classAddressPoint),  
        Description(nullptr), IvarDestroyer(ivarDestroyer) {}  
  
    // The remaining fields are valid only when isTypeMetadata().  
    // The Objective-C runtime knows the offsets to some of these fields.  
    // Be careful when accessing them.  
  
    /// Swift-specific class flags.  
    ClassFlags Flags;  
  
    /// The address point of instances of this type.  
    uint32_t InstanceAddressPoint;  
  
    /// The required size of instances of this type.  
    /// 'InstanceAddressPoint' bytes go before the address point;  
    /// 'InstanceSize - InstanceAddressPoint' bytes go after.  
    uint32_t InstanceSize;  
  
    /// The alignment mask of the address point of instances of this type.  
    uint16_t InstanceAlignMask;  
  
    /// Reserved for runtime use.  
    uint16_t Reserved;  
  
    /// The total size of the class object, including prefix and suffix  
    /// extents.  
    uint32_t ClassSize;  
  
    /// The offset of the address point within the class object.  
    uint32_t ClassAddressPoint;  
  
    // Description is by far the most likely field for a client to try  
    // to access directly, so we force access to go through accessors.  
    const char *Description;  
};
```

TargetClassMetadata继承 TargetAnyClassMetadata

构造方法

Swift特有的标识

实例对象的指针地址

实例对象内存大小

内存对齐mask

保留用于运行时使用

类对象的总大小

类的内存首地址

- TargetClassMetadata** 继承自 **TargetAnyClassMetadata**



逍遥归来 **LV.4**
大前端研发

85

文章

142k

阅读

745

粉丝

关注

私信

目录

收起 ^

类、对象

属性

Swift 进阶二：值类型、引用类型、结...

结构体，值类型

结构体的属性修改问题

结构体的函数调用

方法重载问题

Swift 进阶三：内存分区、方法调度、...

方法调度

方法调用总结

相关推荐

iOS学习栈

378阅读 · 3点赞

【Swift底层】Swift底层探索方法介绍

1.5k阅读 · 3点赞

iOS学习栈

801阅读 · 12点赞

RxSwift 学习笔记

1.0k阅读 · 1点赞

SwiftUI 优点、两大准则、四大数据工...

1.6k阅读 · 1点赞

精选内容

鸿蒙开发：DevEco Studio中的截屏和...

程序员一鸣 · 49阅读 · 3点赞

Android生态震荡：Swift官方支持落地...

Carson带你学An... · 302阅读 · 4点赞

Flutter性能优化Tips

普罗哈基米 · 247阅读 · 8点赞

Swift 官方正式支持 Android，iOS 的跨...

恋猫de小郭 · 3.9k阅读 · 35点赞

找对属于你的技术圈子
回复「进群」加入官方微信群



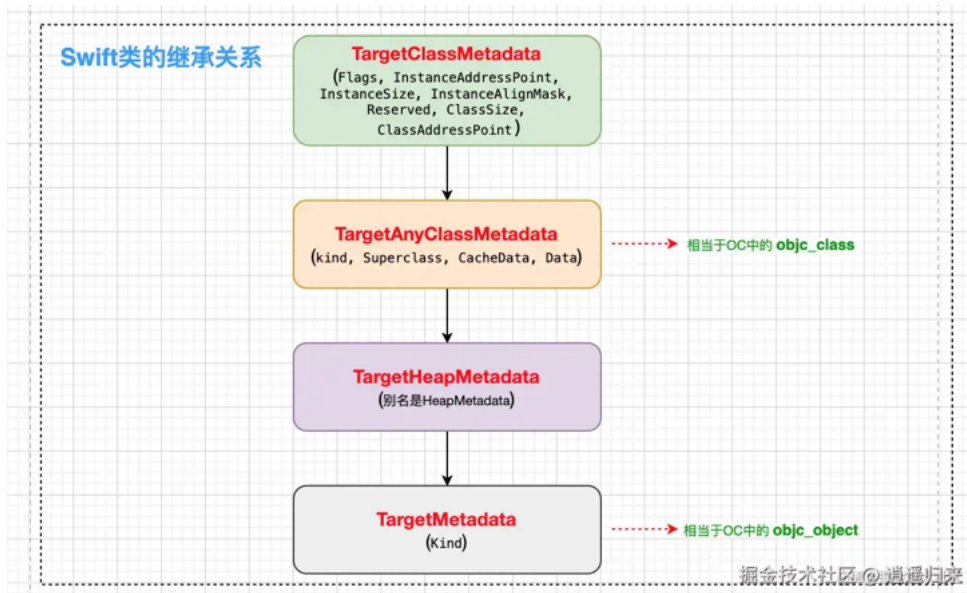
```
template <typename Runtime>
struct TargetAnyClassMetadata : public TargetHeapMetadata<Runtime> {
    using StoredPointer = typename Runtime::StoredPointer;
    using StoredSize = typename Runtime::StoredSize;

    #if SWIFT_OBJC_INTEROP
    constexpr TargetAnyClassMetadata(TargetAnyClassMetadata<Runtime> *isa,
                                      TargetClassMetadata<Runtime> *superclass)
        : TargetHeapMetadata<Runtime>(isa),
          Superclass(superclass),
          CacheData(nullptr, nullptr),
          Data(SWIFT_CLASS_IS_SWIFT_MASK) {}
    #endif
};
```

TargetAnyClassMetadata 继承 TargetHeapMetadata
isa
superclass
方法缓存
存储方法列表等

掘金技术社区 @ 逍遥归来

- 结构与 OC 中的 `objc_class` 的结构一样，有 `isa`，有父类，有 `cacheData`，`Data` 类似于 `objc_class` 中的 `bits`
- 根据上面分析我们可以得到结论：当 `metadata` 的 `kind` 为 `Class` 时，有如下的继承关系：



- 总结，`swift` 的类内存结构可以理解为：

arduino

体验AI代码助手 代码解读 复制代码

```
1 struct Metadata {
2     void *kind;           // 类型标识 (如类、结构体、枚举)
3     void *superClass;     // 父类的 Metadata 指针
4     void *cacheData;      // 方法缓存 (类似 OC 的 cache_t)
5     void *data;           // 指向额外数据的指针
6
7     // 实例布局信息
8     uint32_t flags;        // 类型标志位
9     uint32_t instanceAddressOffset; // 实例变量的起始偏移量
10    uint32_t instanceSize;  // 实例对象占用的内存大小
11    uint16_t instanceAlignMask; // 实例的对齐掩码
12    uint16_t reserved;      // 保留字段
13
14    // 类布局信息
15    uint32_t classSize;     // 类对象占用的内存大小
16    uint32_t classAddressOffset; // 类变量的起始偏移量
17    void *description;      // 类型描述信息
18
19
20    // 类对象与元类对象的关键区别
21    uint32_t flags;        // 包含类型标志位 (如是否为元类)
22    void *vtable;          // 类对象的 vtable 指向实例方法表
23    void *classVtable;     // 元类对象的 vtable 指向类方法表
24
25    // 方法签名表 (所有方法)，存放在类对象中
26    MethodDescriptor* methodDescriptors;
```

```
27     uint32_t methodCount;
28 }
29
30 // 例子:
31 class MyClass {
32     func instanceMethod() {} // 实例方法
33     static func classMethod() {} // 类方法
34 }
35 // 实例方法调用 (通过类对象的 vtable)
36 let obj = MyClass() obj.instanceMethod() // 类对象 -> Metadata -> vtable -> 方法实现
37
38 // 类方法调用 (通过元类对象的 classVtable)
39 MyClass.classMethod() // 类对象 -> 元类对象 -> Metadata -> classVtable -> 方法实现
40
```

属性

- 1. 存储属性：有常量存储属性和变量存储属性两种，都占用内存
- 2. 计算属性：不占用内存，本质为函数。
- 3. 属性观察者：
 - 1. 属性观察可以添加在 类的存储属性 、 继承的存储属性 、 继承的计算属性 中
 - 2. 父类在调用 init 中改变属性值 不会触发 属性观察，子类调用父类的 init 会 触发 属性观察
 - 3. 统一属性在父类和子类都添加观察，在触发观察时：
 - willSet 方法，先子类后父类
 - didSet 方法，先父类后子类
- 4. 延迟属性(lazy)：延迟属性必须有初始（可以为nil），只有在 访问后 内存中才有值，延迟属性对内存有影响， 不能保证线程安全
- 5. 类型属性：类型属性必须有初始值，内存只分配一次，通过 swift_once 函数创建，类似 dispatch_once，是线程安全的。
 - 可以用于单例：

▼

csharp

体验AI代码助手

代码解读

复制代码

```
1 class XXX {
2     static let share: XXX = XXX()
3     private init(){}
4 }
```

Swift 进阶二：值类型、引用类型、结构体

结构体，值类型

▼

csharp

体验AI代码助手

代码解读

复制代码

```
1 struct WSPerson {
2     var age: Int = 18
3 }
4
5 struct WSTeacher {
6     var age: Int
7 }
```

```
struct WSPerson {
    @_hasStorage @_hasInitialValue var age: Int { get set }
    init()
    init(age: Int = 18)
}

struct WSTeacher {
    @_hasStorage var age: Int { get set }
    init(age: Int)
}
```

掘金技术社区 @逍遥归来

- 结构体会自动创建为所有参数赋值的构造函数。
- 结构体开辟的内存存在 **栈区**。
- 结构体的赋值是 **深拷贝**，并且有 **写时复制** 的机制。

结构体的属性修改问题

- 结构体对象 **self** 类型为 **let**，即不可以被修改。
- 结构体中 **函数修改属性**，需要在函数前添加 **mutating** 关键字，本质是给函数的默认参数 **self** 添加了 **inout** 关键字，将 **self** 从 **let** 常量改成了 **var** 变量。
- **mutating** 方法修改结构体属性时，采用的是 "in-place" 的方式，也就是直接在当前实例的内存空间里修改属性值，并没有重新创建一个新的实例来替换原来的实例。这一特性和赋值操作有着本质的区别。

结构体的函数调用

- 值类型对象的函数的调用方式是 **静态调用**，即 **直接地址调用**，调用函数指针，这个函数指针在编译、链接完成后就已经确定了，存放在代码段，而结构体内部并不存放方法。因此可以直接 **通过地址直接调用**

```
44 -> 0x100002b35 <+181>: callq 0x100002b7d ; _2_值类型与引用类型.HZMStack.teach() -> () @掘金技术社区 @逍遥归来
```

这个符号哪里来的？

- 是从 **Mach-O** 文件中的符号表 **Symbol Tables**，但是 **符号表中并不存储字符串**，字符串存储在 **String Table**（字符串表，存放了所有的变量名和函数名，以字符串形式存储），然后根据符号表中的偏移值到字符串中查找对应的字符，然后进行命名重整：**工程名+类名+函数名**

方法重载问题

- 在 **Objective-C** 里，方法重载是不被支持的，不过 **Swift** 却支持，这主要是由它们不同的函数签名机制和语言设计理念造成的。

1. 函数签名机制

- **Objective - C**：它的函数签名 **只依据方法名**，和参数类型没有关系。比如下面这两个方法，在OC看来是一样的，所以无法共存：

objective-c

体验AI代码助手 代码解读 复制代码

```
1 - (void)doSomethingWithInt:(int)value;
2 - (void)doSomethingWithInt:(NSString *)value;
```

- **Swift**：它的函数签名是由 **方法名和参数类型** 共同组成的。下面这样的重载在Swift中是被允许的：

swift

体验AI代码助手 代码解读 复制代码

```
1 func doSomething(value: Int)
2 func doSomething(value: String)
```

2. 消息传递机制

- **OC**：采用的是运行时消息传递机制，方法调用是通过字符串（SEL）来实现的。像 `[obj doSomethingWithInt:1]` 这样的调用，在运行时会被解析为 `SEL @selector(doSomethingWithInt:)`，要是有多同名方法，就会引发冲突。
- **Swift**：使用的是静态 dispatch 机制，在编译时就会确定具体要调用哪个方法。

3. 补充说明

- **Swift 的重载**：除了参数类型不同可以重载外，参数数量不同或者参数标签不同也能实现重载。
- **OC 的替代方案**：在OC中，如果要实现类似功能，通常会采用命名约定，例如 `doSomethingWithInt:` 和 `doSomethingWithString:`。

总结来说，Swift支持方法重载是其类型系统和编译时检查机制的自然结果，而OC不支持则是受限于其动态特性和历史设计。

Swift 进阶三：内存分区、方法调度、指针

方法调度

- Swift 类的方法（非 `final`、非 `static`、非 `@objc` 修饰的）会被存放在一个名为 `vtable` 的表中。
- 只有类能够使用 `vtable`，结构体和枚举由于不支持继承，所以没有 `vtable`

CSS

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 内存布局示例：
2 [实例对象内存]
3   | isa 指针  → [类对象]
4   |             | Metadata 指针  → [Metadata]
5   |             |             | vtable 指针  → [vtable 内存区域]
6   |             |             | 0: init()
7   |             |             | 1: method1()
8   |             |             | 2: method2()
9   |             | 其他类数据...
```

- **方法调用时的流程**，当调用一个类的实例方法时，Swift 运行时会：
 1. 通过实例的 `isa` 指针找到类对象。
 2. 从类对象中获取 `Metadata` 指针。
 3. 从 `Metadata` 中读取 `vtable` 指针。
 4. 根据方法在 `vtable` 中的索引，调用对应的函数实现。
- `vtable` 仅存储 **可重写的方法**，而类的所有方法（包括不可重写的）仍通过元数据（`Metadata`）管理。
- 元类对象（`Metaclass Object`）的 `Metadata` 主要存储类方法（`static / class` 方法）的实现信息。
- 协议方法的签名和实现由 `Witness Table` 管理，与类对象 / 元类对象的 `Metadata` 是分离的。

方法调用总结

- **struct** 是 **值类型**，它的函数调度是 **直接调用**，即 **静态调度**
 - 值类型在函数中如果要修改实例变量的值，则函数前面需要添加 `Mutating` 修饰
- **class** 是 **引用类型**，它的函数调度是通过 `vtable`函数，即 **动态调度**
- **extension** 中的函数是 **直接调用**，即 **静态调度**
- **final** 修饰的函数是 **直接调用**，即 **静态调度**
- **@objc** 修饰的函数是 `methodList` **函数表调度**，如果方法需要在 `OC` 中使用，则类需要继承 `NSObject`
- **dynamic** 修饰的函数调度方式是 `methodList` **函数表调度**，它是动态可以修改的，可以进行 `method-swizzling`
 - `@objc+dynami` 修饰的函数是通过 `objc_msgSend` 来调用的
- 如果函数中的 **参数想要被更改**，则需要在参数的类型前面增加 `inout` 关键字，调用时需要 **传入参数的地**

地址

标签： iOS Swift

评论 0



抢首评，友善交流



0 / 1000 ? 发送

暂无评论数据

- 为你推荐
- SwiftUI 优点、两大准则、四大数据工具、学习路径（2020 读书笔记）

springday 5年前 1.6k 1 评论 Swift
- iOS学习栈（将持续更新）

iOS孜然 5年前 662 4 评论 iOS
- 2023 年 0 基础学习 SwiftUI

SwiftUI做个应用 1年前 5.9k 23 3 SwiftUI
- swift学习笔记《5》- 实用

ytmaylover 6年前 220 点赞 评论 Swift iOS 产品
- Swift 闭包底层原理

给我一个碗我可以走... 1年前 5.5k 13 6 Swift iOS
- 01-🔥Swift5常用核心语法|了解Swift【Swift简介、Swift的版本、Swift编译原理、几种函数派发】

没有故事的Zhang同学 2年前 3.0k 21 8 iOS Swift
- Swift中的 MainActor 底层实现原理

冯志浩 1年前 2.4k 4 2 Swift 掘金·金...
- Modern Concurrency in Swift 的学习笔记

Lebron 3年前 1.3k 5 评论 iOS Swift
- 01-iOS底层原理学习 | 开篇

取个有意思的昵称 2年前 193 点赞 评论 Objective-C

iOS 底层原理探索 之 isa - 类的底层原理结构（下）

阿华12年 4年前 1.2k 17 2 iOS Objecti...

第四届青训营阅读打卡活动来啦，奖品、规则全面升级，快来学习吧

字节跳动青训营 2年前 23k 103 55 前端 大数据 iOS

Swift中闭包底层原理探索

一点田心 4年前 4.5k 24 4 Swift

Swift-学习 sil swift中间语言

路过看风景 4年前 1.3k 1 评论 Swift

OC底层原理系列

空白记忆 4年前 3.3k 11 评论 Objective-C

Swift学习 开发笔记

Si_Simo 6年前 244 点赞 评论 Swift

AI助手